Lect No.(11-13)

Introduction to CSS

What is CSS?

CSS stands for **Cascading Style Sheets** — a powerful **styling language** used to control how HTML elements appear on a web page.

It defines the **look, layout, and design** of web pages — from **colors, fonts, and spacing** to **animations and responsiveness**.

How CSS Works (Behind the Scenes)

1. Browser loads the HTML file.
2. CSS is either **linked**, **embedded**, or **written inline**.
3. CSS rules are **matched to HTML elements**.
4. The **rendering engine** calculates all styles.
5. The final **visual layout** is drawn on the screen.

CSS = Presentation Layer of the Web

When combined with **HTML** and **JavaScript**, it forms the **core web trio**:

Layer	Technology	Responsibility
Structure	HTML	Defines content & structure
Presentation	CSS	Defines style & layout
Behavior	JavaScript	Adds interactivity

Purpose of CSS

✅ To **separate content** (HTML) from **presentation** (style).
✅ To create **consistent design** across multiple pages.
✅ To make web pages **responsive** (fit any screen size).
✅ To improve **user experience** with clean visual design.

Example Without and With CSS

Without CSS:

```
<h1>Hello World</h1>
<p>This is a simple web page.</p>
```

👉 Output: Plain text, no color, no spacing, basic font.

With CSS:

```
<style>
  h1 {
    color: royalblue;
    text-align: center;
  }
  p {
    color: #555;
    font-family: Arial, sans-serif;
    font-size: 18px;
  }
</style>
<h1>Hello World</h1>
<p>This is a styled web page using CSS.</p>
```

👉 Output: Visually appealing page with fonts, colors, and alignment.

Why the Name “Cascading”?

The word **“Cascading”** refers to how **styles are applied in order of priority** (cascade) when multiple rules target the same element.

Priority Order (Highest → Lowest):

1. **Inline CSS**
2. **Internal CSS**
3. **External CSS**
4. **Browser Default Styles**

Example of Cascading:

```
<p style="color:red;">Hello!</p>
```

Even if your stylesheet says: `p { color: blue; }`

👉 The **inline CSS** wins — the text appears **red**.

Key Features of CSS

- Controls design (colors, layout, font, spacing)
- Supports **responsive design** for all devices
- Works with all modern browsers
- Enables **code reusability** through external stylesheets
- Improves performance by **reducing repetitive code**
- Supports **media queries** for adaptive layouts

Real-Life Analogy

- HTML is like a **person's body (structure)**
- CSS is their **clothes and style (appearance)**
- JavaScript is the **person's movement (behavior)**

CSS Syntax

CSS follows a **rule-based syntax** that defines how HTML elements should appear.

Basic Syntax Structure:

```
selector {
  property: value;
}
```

Each rule contains:

1. **Selector** → Targets the HTML element(s)
2. **Property** → The aspect to style (like color, font-size)
3. **Value** → The setting applied to the property

Example:

```
p {
  color: blue;
  font-size: 16px;
  text-align: justify;
}
```

Explanation:

- Targets all `<p>` elements.
- Applies **blue** text color.
- Sets **font size** to 16px.
- Justifies paragraph alignment.

CSS Rule Breakdown Table

Part	Description	Example
Selector	Specifies which element(s) the rule applies to	`p`, `#header`, `.btn`
Property	The feature of the element to style	`color`, `margin`, `border`
Value	Defines how the property should behave	`red`, `10px`, `solid`
Declaration Block	Group of properties within `{ }`	`{color:red; font-size:20px;}`
CSS Rule	A complete set of selector + declarations	`h1 {color:red;}`

Multiple Declarations Example

```
h1 {
  color: #333;
  background-color: #f4f4f4;
  padding: 10px;
  border-radius: 8px;
}
```

Each declaration ends with a **semicolon (;)** — it separates one property from the next.

Formatting Rules (Best Practices)

- ✅ Always use **curly braces** `{}` for declarations.
- ✅ Each **property-value pair** ends with a semicolon `;`.
- ✅ Indent for readability.
- ✅ Use **comments** for explanations.

Example with Comments

```
/* Styling the main title */
h1 {
  color: darkblue;         /* text color */
  text-align: center;      /* center align */
  font-family: Verdana;    /* font style */
}
```

CSS Comments

- **CSS Comments** are **non-executable lines of text** used to describe or annotate your code.
- They are ignored by the browser when rendering the webpage.
- Purpose: To **make CSS more readable, maintainable, and organized** for developers.

Syntax:

```
/* This is a CSS comment */
```

Types of CSS Comments

Technically, CSS has **only one type** of comment, but it can be used in **two ways**:

a) Single-line Comment

Used to describe one specific rule or property.

```
h1 {
  color: blue; /* Heading color */
}
```

b) Multi-line Comment

Used to explain a section or multiple properties.

```
/*
  This section styles the header and navigation bar
  Includes logo alignment and menu design
*/
header {
  background: #222;
  color: white;
}
```

Note:
Unlike JavaScript, CSS does **not** support `//` for single-line comments — only `/* ... */` works.

Uses of CSS Comments

Purpose	Explanation	Example
Code Organization	Divide CSS into clear sections	`/* ===HEADER SECTION=== */`
Documentation	Describe the purpose of a style rule	`/* This sets the main font family */`
Debugging	Temporarily disable a rule	`/* color: red; */`
Team Collaboration	Help teammates understand your code	`/* Added shadow as per design feedback */`

Best Practices

- Keep comments **short and relevant**
- Place them **before code sections**
- Use **uppercase headings** for major sections
- Avoid commenting **every single property**

Example

```
/* ======= MAIN PAGE STYLES ======= */

/* Header */
header {
  background-color: #333;
  color: white; /* Text color */
}

/* Footer */
footer {
  text-align: center;
  padding: 10px;
}
```

Adding CSS to HTML

CSS can be added to HTML in **three main ways**, depending on the project size, maintainability, and purpose.

Inline CSS

- **Definition:** Adding CSS **directly inside an HTML element** using the `style` attribute.
- **Use Case:** Quick testing or small projects.
- **Syntax:** `<tag style="property: value;">Content</tag>`

Example:

```
<p style="color: blue; font-size: 18px;">
  This paragraph is blue and has 18px font size.
</p>
```

✅ Pros

- Quick to apply
- Easy for testing single elements

⚠️ Cons

- Hard to maintain in large projects
- Cannot reuse styles
- Clutters HTML code

Internal CSS (Embedded CSS)

- **Definition:** Adding CSS inside a `<style>` tag in the HTML `<head>` section.
- **Use Case:** Styling a **single HTML page** or when CSS is small.
- **Syntax:**

```
<head>
   <style> selector { property: value;} </style>
</head>
```

Example:

```
<head>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f0f0f0;
    }
```

```
    h1 {
      color: navy;
      text-align: center;
    }
  </style>
</head>
```

✅ Pros

- Keeps CSS in one place for a page
- Good for page-specific styling

⚠️ Cons

- Cannot be reused across multiple pages
- Not ideal for large websites

External CSS

- **Definition:** Writing CSS in a **separate `.css` file** and linking it to HTML using `<link>`.
- **Use Case:** Large projects and **multi-page websites**.
- **Syntax:**

```
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Example (`styles.css`):

```
body {
  font-family: 'Roboto', sans-serif;
  background-color: #f9f9f9;
```

```
  margin: 0;
}

h1 {
  color: #007bff;
  text-align: center;
}
```

✅ Pros

- CSS is **separate** from HTML → cleaner code
- Can be **reused across multiple pages**
- Easy to maintain and scale

⚠️ Cons

- Requires an extra HTTP request (can be minimized with optimization)

Comparison Table

Feature	Inline CSS	Internal CSS	External CSS
Location	Inside HTML element	Inside `<style>` tag	Separate `.css` file
Reusability	Not reusable	Only for one page	Reusable across pages
Maintenance	Hard	Medium	Easy
Best Use	Quick testing	Small single-page projects	Large, multi-page websites

Best Practices

- Prefer **External CSS** for scalability

- Use **Internal CSS** for page-specific overrides
- Avoid **Inline CSS** except for quick testing
- Organize CSS **logically** with comments and section headers
- Keep **file names and folder structures** consistent

Example of All Three Together

```
<!DOCTYPE html>
<html>
<head>
  <title>Adding CSS Example</title>

  <!-- Internal CSS -->
  <style>
    body {
      font-family: Arial;
      background-color: #f4f4f4;
    }
    h1 {
      color: green;
    }
  </style>

  <!-- External CSS -->
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <!-- Inline CSS -->
  <p style="color: red; font-weight: bold;">
    This is a red bold paragraph.
  </p>
  <h1>Welcome to CSS Styling!</h1>
</body>
</html>
```

CSS Selectors

CSS Selectors are **patterns used to select HTML elements** to apply styles. Choosing the correct selector is crucial for **efficient styling, code readability, and maintainability**.

Types of CSS Selectors

1. Universal Selector *

- Selects **all elements** on the page.
- Usually used to **reset or normalize styles**.

Syntax:

```css
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

Explanation:
Removes default margin and padding for all elements.

---

2. Type Selector (Element Selector)

- Selects elements by **tag name**.
- Applies style to **all occurrences** of that element.

Syntax:

```css
p {
  color: blue;
  font-size: 16px;
}
```

Example:
All `<p>` paragraphs on the page will have **blue text**.

---

3. Class Selector `.`

- Selects **elements with a specific class attribute**.
- Can apply to **multiple elements**.

Syntax:

```
.highlight {
  background-color: yellow;
  font-weight: bold;
}
```

HTML Example:

```
<p class="highlight">Highlighted Text</p>
<div class="highlight">Another Highlight</div>
```

Result:
Both elements have a yellow background.

---

4. ID Selector `#`

- Selects an element by **unique ID**.
- Should be **used only once per page**.

Syntax:

```
#header {
  background-color: gray;
  padding: 10px;
}
```

HTML Example:

```
<div id="header">Site Header</div>
```

Result:
Only the element with `id="header"` is styled.

---

5. Group Selector `,`

- Apply **same styles** to multiple selectors at once.

Syntax:

```
h1, h2, h3 {
  color: darkblue;
}
```

Result:
All `<h1>`, `<h2>`, and `<h3>` elements are dark blue.

---

6. Descendant Selector (Space)

- Selects **nested elements** within a parent element.

Syntax:

```
div p {
  color: green;
}
```

HTML Example:

```
<div>
  <p>This paragraph is green.</p>
</div>
<p>This paragraph is NOT green.</p>
```

Result:
Only `<p>` inside `<div>` is styled.

---

7. Child Selector `>`

- Selects **direct children** of an element.

Syntax:

```
div > p {
  color: red;
}
```

HTML Example:

```
<div>
  <p>Direct child styled red.</p>
  <section>
    <p>Nested paragraph not affected.</p>
  </section>
</div>
```

Result:
Only the **direct child** **<p>** is red.

---

8. Adjacent Sibling Selector +

- Selects **next sibling element** immediately after a specific element.

Syntax:

```
h1 + p {
  color: purple;
}
```

HTML Example:

```
<h1>Title</h1>
<p>This paragraph is purple.</p>
<p>This paragraph is NOT purple.</p>
```

---

9. General Sibling Selector ~

- Selects **all siblings** after an element.

Syntax:

```css
h1 ~ p {
  color: orange;
}
```

HTML Example:

```html
<h1>Title</h1>
<p>First paragraph orange.</p>
<p>Second paragraph orange.</p>
```

---

10. Attribute Selector `[attribute]`

- Targets elements **based on attributes or attribute values**.

Syntax Examples:

```css
/* Any input */
input[type] {
  border: 1px solid #000;
}

/* Specific type */
input[type="text"] {
  background-color: #f0f0f0;
}
```

---

11. Pseudo-classes

- Select elements in **special states**.

Selector	Description	Example
`:hover`	When mouse hovers	`a:hover { color: red; }`
`:focus`	When input is focused	`input:focus { border-color: blue; }`

`:first-child`	First child element	`li:first-child {font-weight: bold;}`
`:last-child`	Last child element	`li:last-child { color: green; }`
`:nth-child(n)`	Nth child element	`li:nth-child(2) { color: orange; }`

---

12. Pseudo-elements

- Target **part of an element**.

Selector	Description	Example
`::before`	Inserts content before element	`p::before { content: "Note: "; color: red; }`
`::after`	Inserts content after element	`p::after { content: "✓"; color: green; }`
`::first-letter`	First letter of text	`p::first-letter { font-size: 2em; }`
`::first-line`	First line of text	`p::first-line { font-weight: bold; }`

---

13. Combinators

- Combine multiple selectors for **specific targeting**:

Combinator	Symbol	Example	Meaning
Descendant	(space)	`div p`	All `<p>` inside `<div>`
Child	`>`	`div > p`	Direct `<p>` children of `<div>`
Adjacent Sibling	`+`	`h1 + p`	Next `<p>` immediately after `<h1>`
General Sibling	`~`	`h1 ~ p`	All `<p>` siblings after `<h1>`

Practical Example

```
<!DOCTYPE html>
<html>
<head>
  <style>
    /* Type Selector */
    h1 { color: navy; }

    /* Class Selector */
    .highlight { background-color: yellow; }

    /* ID Selector */
    #header { font-size: 24px; }

    /* Descendant Selector */
    div p { color: green; }

    /* Pseudo-class */
    a:hover { color: red; }

    /* Pseudo-element */
    p::first-letter { font-size: 2em; font-weight: bold; }
  </style>
</head>
<body>
  <h1 id="header">Welcome</h1>
  <div>
    <p class="highlight">This is a highlighted paragraph inside div.</p>
  </div>
  <a href="#">Hover over me!</a>
  <p>This paragraph has a styled first letter.</p>
</body>
</html>
```

Outcome:

- Styles are applied **precisely** based on element type, class, ID, location, or state.